

Trabajo 7: Proyecciones 3D

Cuatro vistas de un mismo cubo en OpenGL

Sergio Andrés Majé Franco

Informática Gráfica y Visualización
Universidad Internacional de La Rioja (UNIR)
28 de mayo de 2026

Índice

1. Introducción	2
2. Planteamiento geométrico	2
2.1. Escena común	2
2.2. Matrices de proyección	2
3. Implementación	3
3.1. Estructura del programa	3
3.2. Funciones de vista	4
3.3. Decisiones de representación	4
3.4. Restricción sobre la posición del ojo	5
4. Resultado	5
4.1. Captura del programa	5
4.2. Funcionamiento	5
5. Limitaciones y fallos detectados	6
6. Rescate de Cálculo Vectorial	6
7. Conclusiones	7
8. Formato de entrega	8

1. Introducción

El Trabajo 7 pide construir una ventana OpenGL con cuatro vistas simultáneas del mismo cubo. Cada vista debe usar una proyección distinta: ortográfica, gabinete, perspectiva simétrica y perspectiva oblicua. Además, el enunciado impone una restricción importante: no se puede mover el ojo desde la posición por defecto $(0, 0, 0)$, por lo que no se permite utilizar `gluLookAt()` ni mecanismos equivalentes.

La solución desarrollada mantiene esa restricción de forma estricta. El cubo es el único objeto que se transforma geométricamente para quedar delante del observador. En consecuencia, la comparación entre proyecciones se apoya en matrices de proyección distintas y en transformaciones de modelo cuidadosamente elegidas para que cada familia de proyección resulte legible sin desplazar la posición del ojo.

2. Planteamiento geométrico

2.1. Escena común

Los cuatro cuadrantes representan exactamente la misma escena:

- un cubo centrado en el origen del modelo;
- ejes cartesianos simples para dar referencia espacial;
- una traslación del objeto sobre el eje $-Z$ para situarlo delante del ojo;
- una pose del cubo adaptada al tipo de proyección cuando resulta necesario para que esa vista sea reconocible.

La escena base siempre es el mismo cubo centrado en el origen del modelo, con caras coloreadas y ejes de referencia. Sin embargo, la transformación de modelo no es idéntica en todos los cuadrantes. En la vista ortogonal se usa una pose axonométrica para que se aprecien tres caras manteniendo el paralelismo. En la vista gabinete se deja el cubo frontal para que la cara delantera conserve magnitud y sea la cizalla la que produzca la profundidad oblicua. En las dos perspectivas se mantiene prácticamente la misma colocación del cubo, de modo que la diferencia entre ambas la produzca sobre todo el *frustum*.

2.2. Matrices de proyección

Las matrices se codifican de forma explícita en `main.c`. No se delega la construcción de la cámara a utilidades externas. Las cuatro familias empleadas son las siguientes:

Proyección ortográfica. Se utiliza la forma clásica de la matriz ortográfica:

$$O = \begin{bmatrix} \frac{2}{r-l} & 0 & 0 & -\frac{r+l}{r-l} \\ 0 & \frac{2}{t-b} & 0 & -\frac{t+b}{t-b} \\ 0 & 0 & -\frac{2}{f-n} & -\frac{f+n}{f-n} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Su efecto visual es conservar paralelas todas las aristas del cubo, sin convergencia hacia puntos de fuga.

Proyección gabinete. La vista gabinete se construye combinando una proyección ortográfica con una cizalla sobre el eje de profundidad. En el código se implementa como el producto $O \cdot S$, donde S reproduce la forma conceptual

$$x' = x + kz \cos(\alpha), \quad y' = y + kz \sin(\alpha)$$

con $k = 0,5$ y $\alpha = 45^\circ$. Dado que el cubo se sitúa delante del observador sobre $z < 0$, la cizalla se ajusta de signo para que la profundidad retroceda visualmente en diagonal sobre la imagen. La cara frontal se deja prácticamente en verdadera magnitud, que es precisamente lo que distingue al gabinete de una perspectiva.

Perspectiva simétrica. Se usa una matriz de *frustum* centrada:

$$P = \begin{bmatrix} \frac{2n}{r-l} & 0 & \frac{r+l}{r-l} & 0 \\ 0 & \frac{2n}{t-b} & \frac{t+b}{t-b} & 0 \\ 0 & 0 & -\frac{f+n}{f-n} & -\frac{2fn}{f-n} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

En este caso se cumple $l = -r$ y $b = -t$, de modo que el volumen de visión es simétrico.

Perspectiva oblicua asimétrica. Para la cuarta vista se reaprovecha la misma familia de matriz de *frustum*, pero con límites laterales y verticales no simétricos. En particular, el código fija valores con $l \neq -r$ y $b \neq -t$, de modo que el volumen visible queda desplazado respecto al eje óptico. Así aparece una perspectiva *off-axis*, claramente diferente de la simétrica, sin mover el ojo.

3. Implementación

3.1. Estructura del programa

Todo el trabajo queda contenido en `main.c`, como pedía el enunciado. La lógica principal se divide en cuatro bloques:

- creación de la ventana X11 y del contexto GLX;

- construcción explícita de matrices ortográficas, de cizalla y de *frustum*;
- renderizado del cubo en cuatro *viewports* mediante funciones separadas por proyección;
- gestión de etiquetas, redimensionado y eventos de cierre.

El uso de *viewports* permite reutilizar la misma geometría sin duplicar escenas. Cada cuadrante limpia su propio fondo, configura su propia proyección, reinicia la matriz `GL_MODELVIEW` y dibuja el cubo dentro de un sub-área con márgenes. Después se dibujan divisores y etiquetas en *overlay* para reforzar la lectura visual en la captura.

3.2. Funciones de vista

Para mantener la implementación defendible y evitar contaminación entre matrices, el código se organiza con una función por vista:

- `drawOrthogonalView()`, que usa `glOrtho` y una pose axonométrica;
- `drawCabinetView()`, que usa una matriz ortográfica compuesta con cizalla;
- `drawSymmetricPerspectiveView()`, que usa un *frustum* centrado;
- `drawObliquePerspectiveView()`, que usa un *frustum* asimétrico u *off-axis*.

Cada una de esas funciones llama a `glViewport()`, carga su proyección, reinicia `GL_MODELVIEW` con `glLoadIdentity()` y aplica únicamente transformaciones del objeto. El uso de `glPushMatrix()` y `glPopMatrix()` evita que una vista contamine a la siguiente.

3.3. Decisiones de representación

Se han tomado varias decisiones para que la comparación entre proyecciones resulte clara:

- las caras del cubo están coloreadas para distinguir su orientación;
- las aristas se dibujan encima de las caras para reforzar la percepción de volumen;
- los ejes se añaden como referencia espacial mínima;
- cada cuadrante usa un fondo ligeramente distinto, pero muy cercano, para separar visualmente las vistas sin distraer;
- las vistas perspectiva simétrica y perspectiva oblicua comparten casi la misma pose del cubo, para que la diferencia se deba al *frustum* y no a una simple rotación extra.

3.4. Restricción sobre la posición del ojo

El código no usa `gluLookAt()` ni ninguna rutina equivalente de cámara. La escena parte siempre del ojo por defecto y el cubo es el que se traslada hacia el eje $-Z$. Esto responde de forma directa a la condición explícita del trabajo.

4. Resultado

4.1. Captura del programa

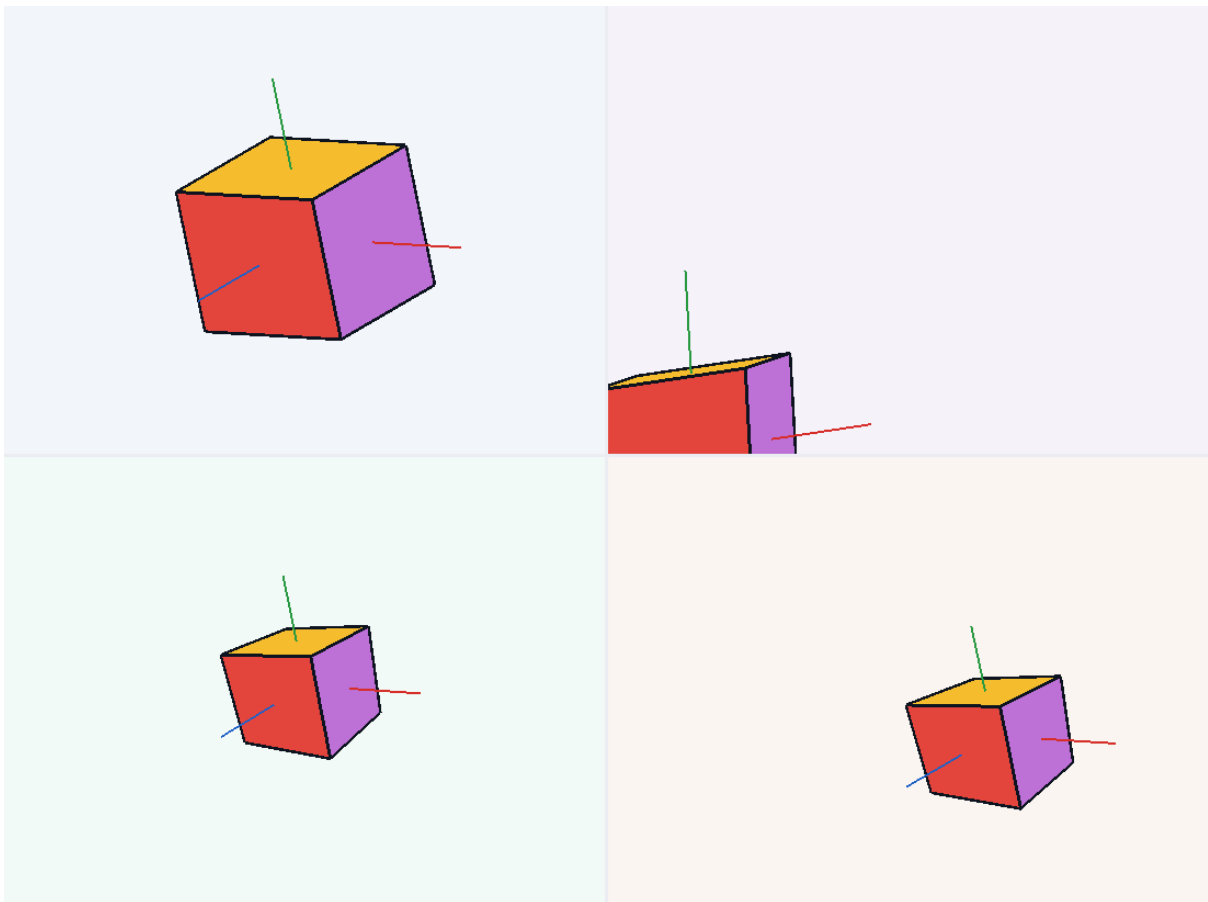


Figura 1. Ventana del Trabajo 7 con las cuatro proyecciones simultáneas.

4.2. Funcionamiento

¿Funciona bien tu programa?

Sí. El programa abre una única ventana OpenGL y muestra de forma estable las cuatro vistas solicitadas. La escena se mantiene coherente durante el redimensionado y cada cuadrante presenta un comportamiento visual compatible con la familia de proyección que le corresponde. La ortográfica mantiene el paralelismo, la gabinete conserva una cara frontal clara y profundidad oblicua reducida, la perspectiva simétrica produce convergencia centrada y la perspectiva oblicua desplaza el volumen de visión de forma reconocible.

5. Limitaciones y fallos detectados

¿Qué fallos existen?

La solución cumple el objetivo del trabajo, pero conviene dejar documentadas varias limitaciones:

- depende de un entorno gráfico con X11/GLX y OpenGL clásico, por lo que no es una solución nativa para Windows sin WSL o sin un servidor X;
- el rotulado usa texto bitmap sencillo generado con `glXUseXFont`, suficiente para la práctica, pero poco flexible desde el punto de vista tipográfico;
- los valores numéricos elegidos para cada matriz se han ajustado manualmente para priorizar legibilidad sobre generalidad;
- la perspectiva oblicua se resuelve como un *frustum* asimétrico, que es una interpretación defendible del requisito, aunque el enunciado no fija una parametrización única para esa cuarta proyección.

6. Rescate de Cálculo Vectorial

Como ampliación del repositorio se ha incorporado un proyecto independiente. Su ubicación es `extras/calculo-vectorial-figura-1/`. Este extra no forma parte de la entrega principal del Trabajo 7, pero recupera la Figura 1 del laboratorio previo de Cálculo Vectorial dentro del mismo entorno técnico de OpenGL clásico.

La superficie representada viene dada por la ecuación:

$$\frac{x^2}{9} + \frac{(y-3)^2}{4} - \frac{(z+1)^2}{9} = 1$$

Se trata de un hiperboloide de una hoja centrado en $(0, 3, -1)$ y orientado a lo largo del eje z . En el programa se parametriza mediante funciones hiperbólicas:

$$x = 3 \cosh(v) \cos(u), \quad y = 3 + 2 \cosh(v) \sin(u), \quad z = -1 + 3 \sinh(v)$$

con $u \in [0, 2\pi]$ y un rango acotado de v para muestrear una banda visible de la superficie.

Desde el punto de vista de OpenGL, la construcción se ha hecho en varias etapas concretas. Primero se fija una proyección perspectiva sencilla mediante una matriz de *frustum*, suficiente para dar profundidad a la escena sin introducir una cámara compleja. Después, en la matriz de modelo se aplica una traslación para separar la superficie del ojo y dos rotaciones fijas para mostrar simultáneamente su apertura, su cintura central y su desplazamiento respecto del origen global.

La geometría no se almacena como una malla indexada, sino que se genera de forma procedural en tiempo de renderizado. El programa recorre una rejilla paramétrica discreta con dos familias de curvas:

- curvas de parámetro v constante, que producen anillos elípticos alrededor del eje de la superficie;
- curvas de parámetro u constante, que producen generatrices que ascienden y descienden por el hiperboloide.

Cada punto de esa rejilla se evalúa sustituyendo (u, v) en la parametrización anterior y enviando el resultado a OpenGL con `glVertex3d`. Para hacer visible la estructura matemática, se usan `GL_LINE_STRIP` sucesivos en lugar de polígonos rellenos. Eso da lugar a una malla alámbrica donde se aprecia bien la doble curvatura de la superficie y su carácter reglado en sentido visual.

Además, se dibujan ejes cartesianos con colores distintos antes de la malla. Esos ejes ayudan a leer que el hiperboloide rescatado no está centrado en el origen, sino trasladado al entorno de $(0, 3, -1)$. En conjunto, esta implementación prioriza claridad geométrica y ligereza de código frente a una representación sombreada más costosa.

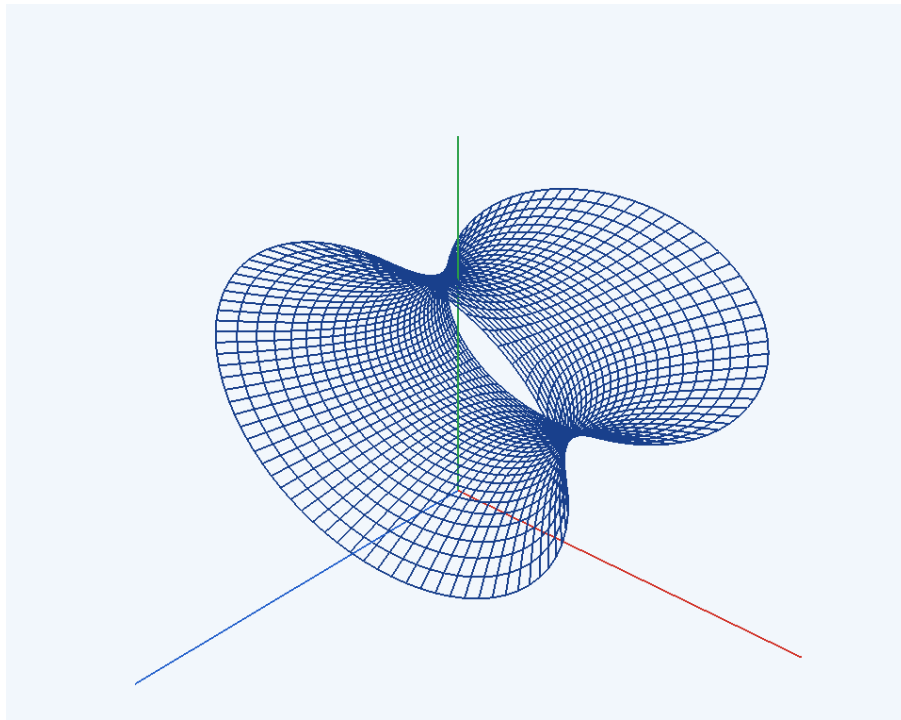


Figura 2. Hiperboloide de una hoja rescatado de Cálculo Vectorial.

Este rescate aporta valor al repositorio porque reutiliza la misma infraestructura gráfica de la asignatura para representar otra superficie relevante, pero sin mezclarla con la práctica principal.

7. Conclusiones

El trabajo resuelve de forma directa la comparación entre distintas proyecciones 3D sobre una misma geometría, que es precisamente el foco del Tema 7. La implementación es sencilla de defender porque separa con claridad la escena común de las matrices de proyección y evita dependencias de cámara que contradigan el enunciado.

Además, la nueva estructura del repositorio deja el laboratorio del reloj como antecedente histórico, incorpora el Trabajo 7 como práctica principal y añade un extra matemático reutilizable. El resultado es un monorepo más ordenado, más fácil de mantener y con mejor continuidad técnica entre ejercicios.

8. Formato de entrega

El informe se mantiene en LaTeX y se genera a PDF dentro del repositorio. Esto deja documentado el trabajo de forma reproducible, aunque la nota del enunciado indicaba expresamente que sólo se aceptaban formatos editables y no PDF.